

궤도결정과 필터링 HW#2

2022311224 이진진

1) Explain the POD process using the differential correction.

POD(Precise orbit determination) 과정이란 궤도를 cm 단위까지 매우 정밀하게 결정하는 것을 말하며 계산된 관측 값과 실제 관측 값의 차이(residual)를 줄이는 것이 최종목표이다. 그 중 Non-linear least square을 통한 differential correction을 사용하는 것이 이를 가장 효과적으로 수행할 수 있는 방법이다. 이 방법은 Nominal state의 초기 값(y_{n0})을 추정하고 동일한 시간에 대해 반복적으로 differential을 거쳐 오차를 최소화하는 것이다.

A sensor can measure **range(ρ)**, **azimuth(β)** and **elevation(el)** at each time t_i

$$y_{0i} = \begin{bmatrix} \rho_0 \\ \beta_0 \\ el_0 \end{bmatrix} \text{ at } t_i \quad \text{ti 시간에서의 Range, Azimuth, Elevation을 설정}$$

We need to compute predicted values

$$y_{ci} = \begin{bmatrix} \rho_c \\ \beta_c \\ el_c \end{bmatrix} \text{ at } t_i \quad y_{ci}; \text{ a non-linear function of } \vec{r}, \vec{v}$$

초기시간 t_0 에 대한 dynamic model을 통해 초기 State X를 나타낼 수 있다.

state $X = (\vec{r}(t_0), \vec{v}(t_0))$, at initial time $\vec{r}(t_0), \vec{v}(t_0)$ are given
dynamic model $f(\vec{r}(t_0), \vec{v}(t_0))$

$$\Rightarrow \vec{r}(t_i), \vec{v}(t_i) \Rightarrow \text{measurement model, } y(\vec{r}(t_i), \vec{v}(t_i)) \Rightarrow \begin{bmatrix} \rho_c \\ \beta_c \\ el_c \end{bmatrix} = y_{ci} \text{ at } t_i$$

같은 시간 t_i 에서 관측을 하고 이 값으로 Non-linear function 방법을 이용한 Measurement model을 y_{ci} 로 나타낼 수 있다.

We use a first-order Taylor series expansion about a nominal trajectory.

$$y_{ci} = y_{ni} + \Delta r_i \frac{\partial y_{ni}}{\partial r_i} + \Delta r_j \frac{\partial y_{ni}}{\partial r_j} + \Delta r_k \frac{\partial y_{ni}}{\partial r_k} + \Delta v_i \frac{\partial y_{ni}}{\partial v_i} + \Delta v_j \frac{\partial y_{ni}}{\partial v_j} + \Delta v_k \frac{\partial y_{ni}}{\partial v_k}$$

$$\text{where } \frac{\partial y_{ni}}{\partial r_i} = \frac{\partial y_{ci}}{\partial r_i} \bigg|_{r_i, v_i} \quad \text{Taylor series를 통해 이 차이를 계속 줄여나간다.}$$

The nominal trajectory is

$$y_{ni} = y(\vec{r}_n(t_i), \vec{v}_n(t_i))$$

The residuals $\vec{r}_i$ residual은 계산된 값과 실제 값의 차이를 말하며 다음과 같이 정의한다.

$$\begin{aligned} \vec{r}_i &= y_{0i} - y_{ci} \\ &= y_{0i} - \left[y_{ni} + \Delta r_i \frac{\partial y_{ni}}{\partial r_i} + \Delta r_j \frac{\partial y_{ni}}{\partial r_j} + \Delta r_k \frac{\partial y_{ni}}{\partial r_k} + \Delta v_i \frac{\partial y_{ni}}{\partial v_i} + \Delta v_j \frac{\partial y_{ni}}{\partial v_j} + \Delta v_k \frac{\partial y_{ni}}{\partial v_k} \right] \end{aligned}$$

Assume each measurement is weighted using its standard deviation

$$w_\rho = \frac{1}{\sigma_\rho}, \quad w_\beta = \frac{1}{\sigma_\beta}, \quad w_{el} = \frac{1}{\sigma_{el}} \quad \text{각 관측 값에 대해 가중치(w)를 부여한다.}$$

The cost function

$$J = \sum_{i=1}^N (w_i \bar{r}_i)^T (w_i \bar{r}_i) \quad \text{residual}(r) \text{을 통해 표현되는 Cos function}(J) \text{를 최소화 해야한다.}$$

$$\text{where } w_i = \begin{bmatrix} w_p & 0 & 0 \\ 0 & w_\beta & 0 \\ 0 & 0 & w_d \end{bmatrix}, \quad i=1,2,\dots,N$$

Set the first derivatives of J equals to zero.

$$\sum_{i=1}^N w_i^T w_i \bar{r}_i \frac{\partial \bar{r}_i}{\partial \vec{v}} = 0 \quad \text{이를 위해 미분한 값이 0이 되는 지점을 찾아야한다.}$$

$$\frac{\partial \bar{r}_i}{\partial \vec{v}} = \begin{bmatrix} \frac{\partial \bar{r}_i}{\partial v_1} \\ \frac{\partial \bar{r}_i}{\partial v_j} \\ \frac{\partial \bar{r}_i}{\partial v_k} \end{bmatrix}, \quad \sum_{i=1}^N \frac{\partial \bar{r}_i}{\partial \vec{v}} \Rightarrow \begin{bmatrix} \frac{\partial \bar{r}_1}{\partial v_1} & \frac{\partial \bar{r}_2}{\partial v_1} & \dots & \frac{\partial \bar{r}_N}{\partial v_1} \\ \frac{\partial \bar{r}_1}{\partial v_j} & \frac{\partial \bar{r}_2}{\partial v_j} & \dots & \frac{\partial \bar{r}_N}{\partial v_j} \\ \frac{\partial \bar{r}_1}{\partial v_k} & \frac{\partial \bar{r}_2}{\partial v_k} & \dots & \frac{\partial \bar{r}_N}{\partial v_k} \end{bmatrix} \approx - \begin{bmatrix} \frac{\partial y_{c1}}{\partial v_1} & \frac{\partial y_{c2}}{\partial v_1} & \dots & \frac{\partial y_{cN}}{\partial v_1} \\ \frac{\partial y_{c1}}{\partial v_j} & \frac{\partial y_{c2}}{\partial v_j} & \dots & \frac{\partial y_{cN}}{\partial v_j} \\ \frac{\partial y_{c1}}{\partial v_k} & \frac{\partial y_{c2}}{\partial v_k} & \dots & \frac{\partial y_{cN}}{\partial v_k} \end{bmatrix}$$

$$\therefore -1 \begin{bmatrix} \frac{\partial y_{c1}}{\partial r_1} & \frac{\partial y_{c2}}{\partial r_1} & \dots & \frac{\partial y_{cN}}{\partial r_1} \\ \frac{\partial y_{c1}}{\partial r_j} & \ddots & & \vdots \\ \vdots & & \ddots & \frac{\partial y_{cN}}{\partial v_j} \\ \frac{\partial y_{c1}}{\partial v_k} & \dots & \frac{\partial y_{cN-1}}{\partial v_k} & \frac{\partial y_{cN}}{\partial v_k} \end{bmatrix} \begin{bmatrix} w_1^2 \bar{r}_1 \\ w_2^2 \bar{r}_2 \\ \vdots \\ w_N^2 \bar{r}_N \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \text{위의 식을 정리한 결과이다}$$

$$\Leftrightarrow -A^T W \begin{bmatrix} \bar{r}_1 \\ \bar{r}_2 \\ \vdots \\ \bar{r}_N \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$\text{where } W = \begin{bmatrix} w_1^2 & 0 & \dots & 0 \\ 0 & \ddots & & \vdots \\ \vdots & & \ddots & 0 \\ 0 & \dots & 0 & w_N^2 \end{bmatrix}$$

위의 식을 두 항으로 분리하여
계산한다.

$$-A^T W \begin{bmatrix} y_{01} - y_{n1} \\ y_{02} - y_{n2} \\ \vdots \\ y_{0N} - y_{nN} \end{bmatrix} - A^T W \begin{bmatrix} \frac{\partial y_{n1}}{\partial r_1} & \frac{\partial y_{n1}}{\partial r_j} & \dots & \frac{\partial y_{n1}}{\partial v_k} \\ \frac{\partial y_{n2}}{\partial r_1} & \ddots & & \vdots \\ \vdots & & \ddots & \frac{\partial y_{nN-1}}{\partial v_k} \\ \frac{\partial y_{nN}}{\partial r_1} & \dots & \frac{\partial y_{nN}}{\partial v_j} & \frac{\partial y_{nN}}{\partial v_k} \end{bmatrix} \begin{bmatrix} \Delta r_1 \\ \Delta r_j \\ \Delta r_k \\ \Delta v_1 \\ \Delta v_j \\ \Delta v_k \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

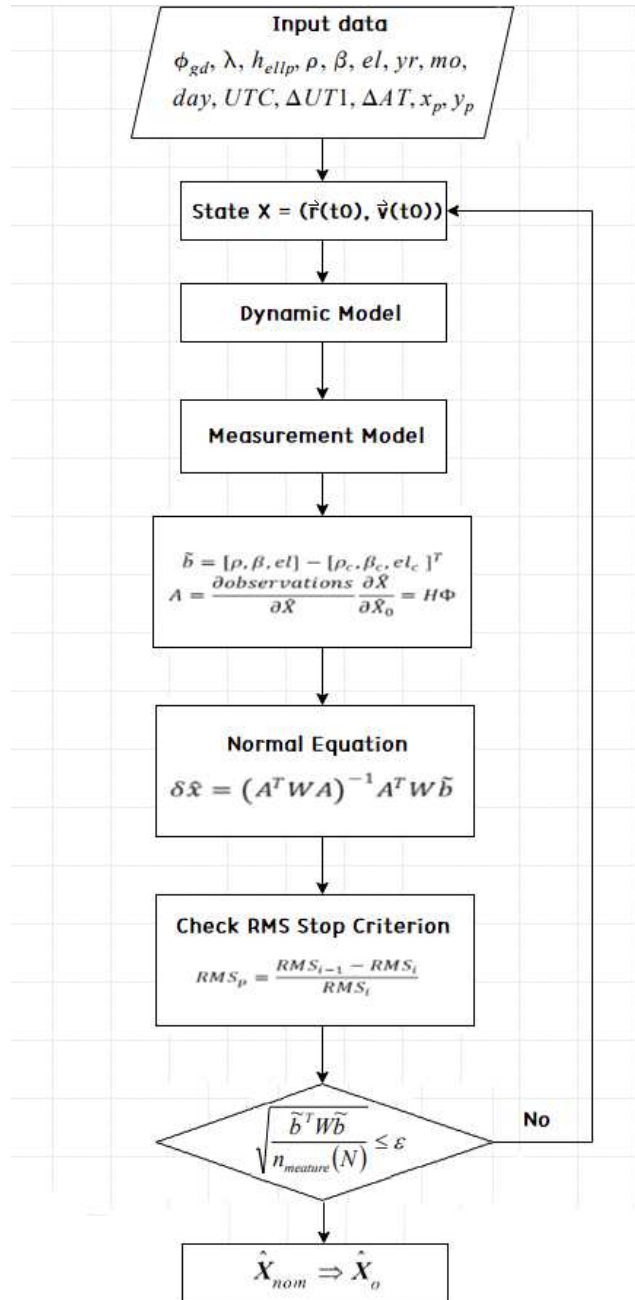
$$A^T W \tilde{b} - A^T W A \delta \hat{X} = 0$$

$$\therefore \delta \hat{X} = (A^T W A)^{-1} A^T W \tilde{b} \quad \text{이를 간단히 나타낸 식이다.}$$

$$X_{new} = X_{old} + \delta \hat{X}$$

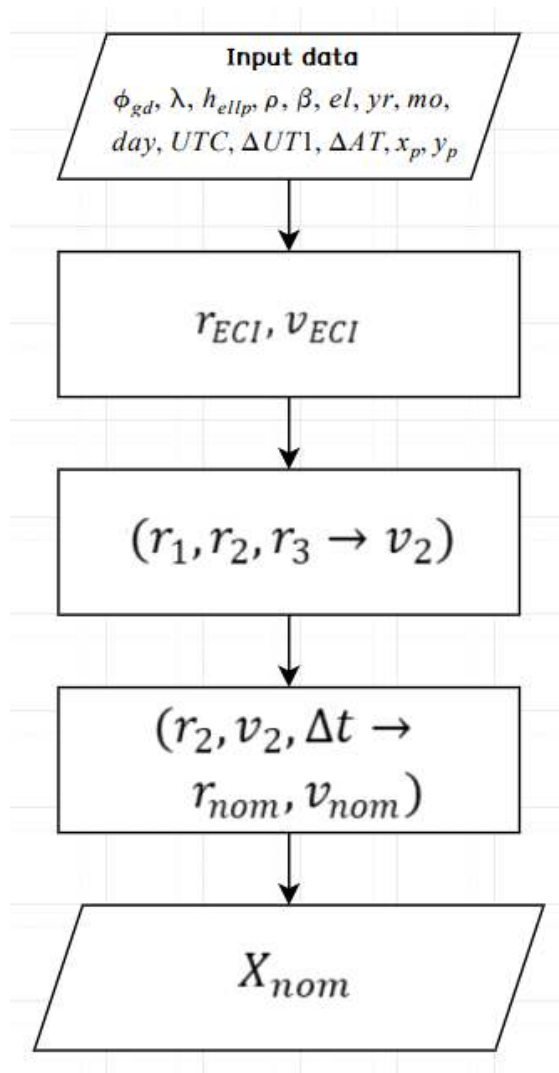
$$\text{where } X = \begin{Bmatrix} \vec{r}(t_0) \\ \vec{v}(t_0) \end{Bmatrix} \text{ at epoch time}$$

따라서 위의 식에서 RMS 값이 사용자가 설정한 범위 이하가 되면 epoch time에 수렴한다고 할 수 있으며 임의의 시간에 대한 위치와 속도를 구할 수 있다.



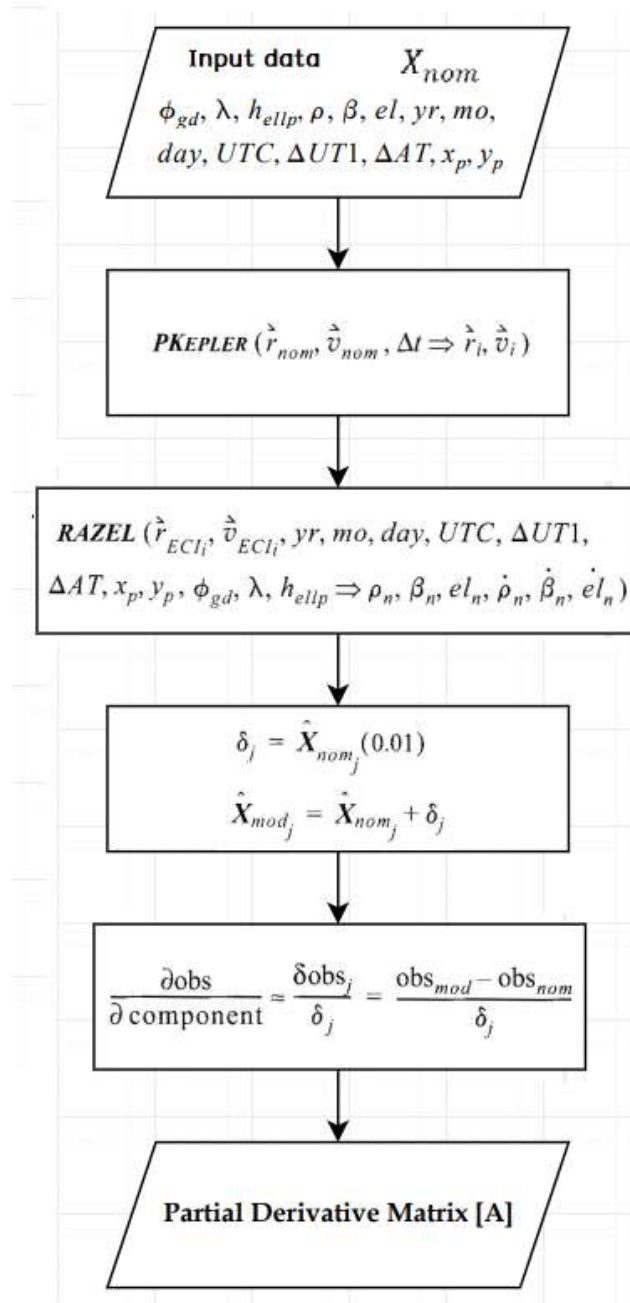
2) Explain how to calculate the nominal state

nominal state를 계산하기 위하여 문제 1)의 flowchart에서 입력한 input data를 이용해 ECI 좌표계로 관측 데이터를 변환한다. SEZ좌표계에서의 관측 값(ρ, β, el)을 이용하여 위치와 속도 벡터를 계산할 수 있다. 이 과정은 `razel2rv` function을 사용하여 변환을 수행할 수 있다. 여러 번의 과정을 거쳐 얻은 값들 중에 임의로 3개의 데이터를 선정하면 Herrick-Gibbs 방법을 이용할 수 있으며 가운데 데이터인 r_2 와 v_2 를 계산할 수 있게 된다. 이 값을 PKepler식으로 전파하면 r_2, v_2 와 epoch time을 최종적으로 nominal state의 위치와 속도를 구할 수 있다. 이 과정을 여러 차례 반복하면 nominal state vector를 구하게 된다.

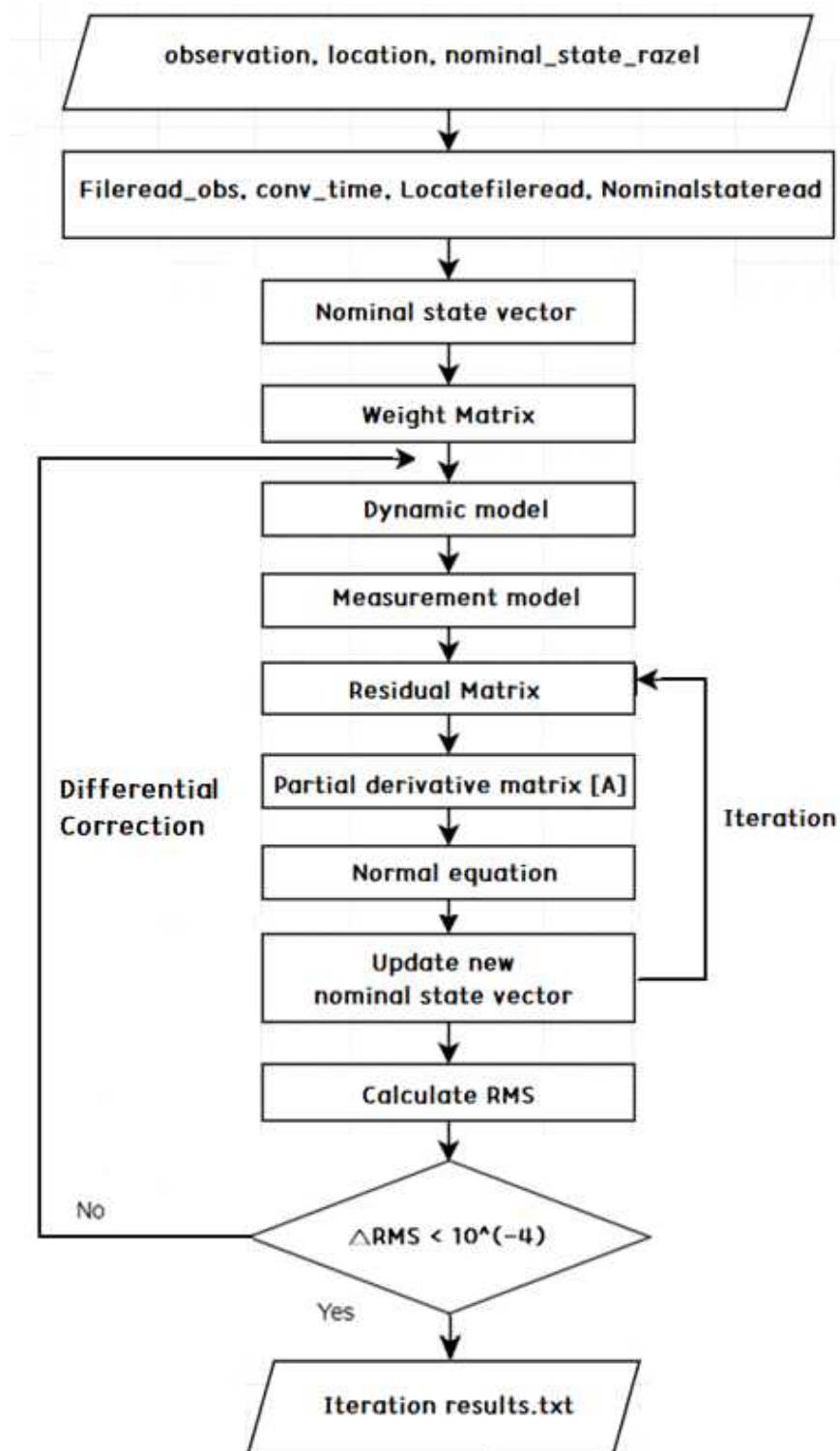


3) Explain how to calculate the partial derivative matrix (A)

Partial derivative matrix A는 하나의 epoch time에서의 nominal state가 매 시간 바뀌기 때문에 초기 상태에서의 값이 어떻게 바뀌는지를 판단하기 위한 것이다. 앞의 문제에서 이용한 input data에 $\hat{X}_{nom}$ 을 추가하여 입력한다. 이를 Pkepler 식을 통해 t_i 시간에서의 nominal state를 구한 후에 이 값의 미세한 차이(ex. 0.01)를 더하여 nominal state를 수정해 나간다. 이를 $\hat{X}_{mod}$ (modified nominal state)로 표현하였다. 위와 같은 과정으로 (ρ, β, el) 를 변환하고 마지막으로 A matrix의 각 항을 계산하면 된다.



4) Draw a flowchart for all the codes provided.



5) Analyze the program codes provided as detail as possible. Draw a flowchart for each code. Make a document for each code.

① fileread_obs

observation - Windows 메모장

위성 이름	년	월	일	시	분	초	ρ	β	el
7734	1995	1	29	02	38	37.000	2047.50200	60.4991	16.1932
7734	1995	1	29	02	38	49.000	1984.67700	62.1435	17.2761
7734	1995	1	29	02	39	02.000	1918.48900	64.0566	18.5515
7734	1995	1	29	02	39	14.000	1859.32000	65.8882	19.7261
7734	1995	1	29	02	39	26.000	1802.18600	67.9320	20.9351
7734	1995	1	29	02	39	38.000	1747.29000	70.1187	22.1319
7734	1995	1	29	02	39	50.000	1694.89100	72.5159	23.3891
7734	1995	1	29	02	40	03.000	1641.20100	75.3066	24.7484
7734	1995	1	29	02	40	15.000	1594.77000	78.1000	25.9799
7734	1995	1	29	02	40	27.000	1551.64000	81.1197	27.1896
7734	1995	1	29	02	40	39.000	1512.08500	84.3708	28.3560
7734	1995	1	29	02	40	51.000	1476.41500	87.8618	29.4884
7734	1995	1	29	02	41	03.000	1444.91500	91.5955	30.5167
7734	1995	1	29	02	41	15.000	1417.88000	95.5524	31.4474
7734	1995	1	29	02	41	27.000	1395.56300	99.7329	32.2425
7734	1995	1	29	02	41	39.000	1378.20200	104.0882	32.8791
7734	1995	1	29	02	41	51.000	1366.01000	108.6635	33.3788
7734	1995	1	29	02	42	03.000	1359.10000	113.2254	33.5998

Ln 1, Col 1 100% Windows (CRLF) UTF-8

fileread_obs.m 파일은 관측정보 및 관측 시간에 대한 정보를 읽는 함수이다. observation.txt 파일에 왼쪽과 같은 정보가 기록되어 있고 이를 fileread_obs에서 읽어 들인다. 각각의 과정에 대한 flowchart는 아래 그림과 같다.

```

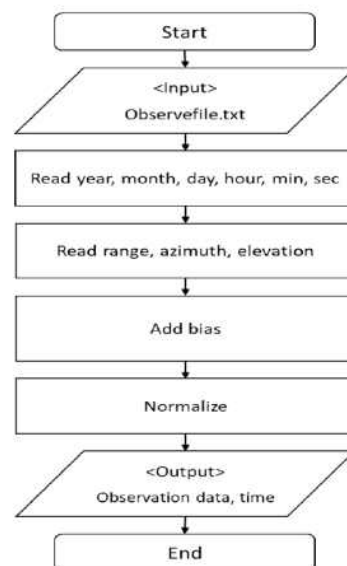
function [satn rho rho_normalize az el date]
= fileread_obs(locatefile)
dat = load(locatefile);

name = dat(:,1);
date(:,1) = dat(:,2);
date(:,2) = dat(:,3);
date(:,3) = dat(:,4);
date(:,4) = dat(:,5);
date(:,5) = dat(:,6);
date(:,6) = dat(:,7);

range = dat(:,8);
azi = dat(:,9);
ele = dat(:,10);

satn = name; observation.txt의 정보를 분류
% add bias
rho = (range-80) * 1000; range에 bias를 추가
% Normalize
Re = 6378.137; % km 지구의 반지름을 고려하여
rho_normalize = range/Re; rho, beta, el을 normalize
az = (azi-0.0081)*(pi/180);
el = (ele-0.0045)*(pi/180);

end observation data와 time을 output
  
```



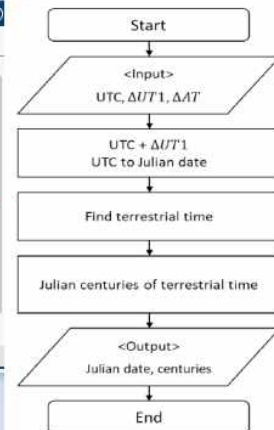
② conv_time

conv_time.m 파일은 관측 값을 [년 월 일 시 분 초] 6개의 함수에 대하여 순서대로 1, 2, 3, 4, 5, 6으로 구분하여 Julian date 및 centuries로 변환한다.

```

conv_time.m
function [ut1_JD, tt_JD, ttt]=conv_time(utc,del_at,del_ut1)
%#eml
ut1=utc;
ut1(:,6) = ut1(:,6) + del_ut1; % utc: [yr mon day hour minute second]
ut1_JD = JD(ut1); % UTC = UT1 - del_UT1
% utc에서 오차값을 적용하여 Julian으로 변환 (여기서는 :.6이므로 second)
tt = utc;
tt(:,6) = tt(:,6) + del_at + 32.184; % tt: Terrestrial time
tt_JD = JD(tt); % TAI = UTC + del_at, TT = TAI + 32.184
% Terrestrial time(지구시)에 오차값을 더하여 Julian으로 변환
ttt = (tt_JD-2451545.0)/36525; % ttt: Julian centuries of tt
% Julian centuries 값 출력

```



③ nominalstateread

```

*nominal_state_razel - Win...
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)
Observation Data Type : RAZEL
R(1) R(2) R(3) V(1) V(2) V(3) JDUT1
6094689.230031 ri
2811933.816635 rj
2631660.787750 rk
3619.616960 vi
-2043.225662 vj
-6190.325113 vk
2449746.610154 julian date
Ln 5 100% Windows (CRLF) UTF-8

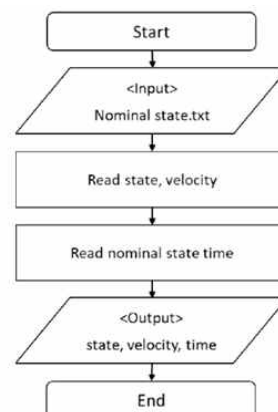
```

nominalstateread.m 파일은 Nominal state의 데이터를 읽는 함수이다. nominal_state_razel.txt 파일에 왼쪽 사진과 같이 위치와 속도 그리고 Julian date에 대한 7개의 값이 기록되어 있으며 이를 nominalstateread.m에서 읽어 들인다. 각각의 과정에 대한 flowchart는 아래 그림과 같다.

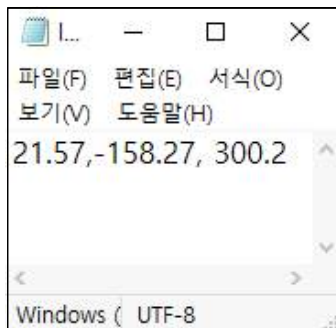
```

nominalstateread.m
function [r v jdut1 filetype] = nominalstateread(file)
r = zeros(1,3);
v = zeros(1,3); % 위치와 속도에 대해 각각 1x3, 3x1의 0행렬로 표현
fp = fopen(file,'r'); % 읽기 위한 파일을 open
s1 = fgetl(fp); % fp의 새 줄 문자를 제거하여 파일의 라인을 반환
filetype = sscanf(s1,'Observation Data Type : %s'); % 문자형 벡터를 숫자의 열 벡터로 반환
gab = fgetl(fp);
data = fgetl(fp);
rv = sscanf(data,'%f %f %f %f %f %f %f',[1,7]);
r(:) = rv(1:3); % ri, rj, rk
v(:) = rv(4:6); % vi, vj, vk
jdut1 = rv(7); % julian date
fclose(fp)
end

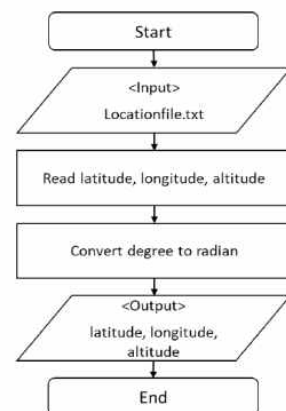
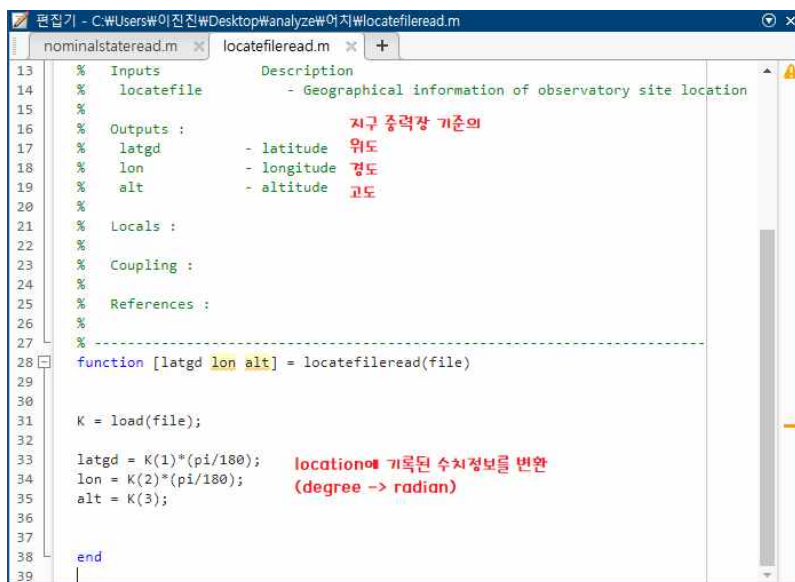
```



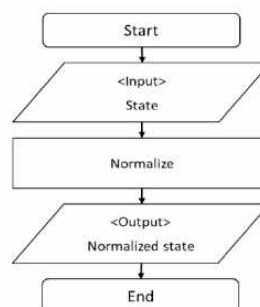
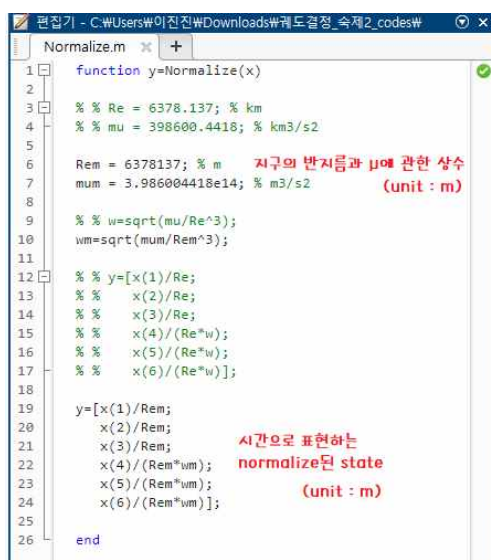
④ locatefileread



locatefileread.m파일은 관측 지점의 위도, 경도, 고도를 읽는 함수이다. location.txt 파일에 왼쪽과 같이 기록된 수치에 따라 locatefileread.m에서 이를 읽어 들이고 실행한다. 단위 변환을 위해 $\pi/180$ 을 각각 곱하여 radian unit으로 맞추었다. K라는 함수로 세 가지의 element를 나타내었고 이 과정에 대한 flowchart는 아래 그림에서 확인할 수 있다.



⑤ Normalize



지구의 반경과 중력 상수를 이용한 normalize 작업이다. 단위는 m로 변환했다.

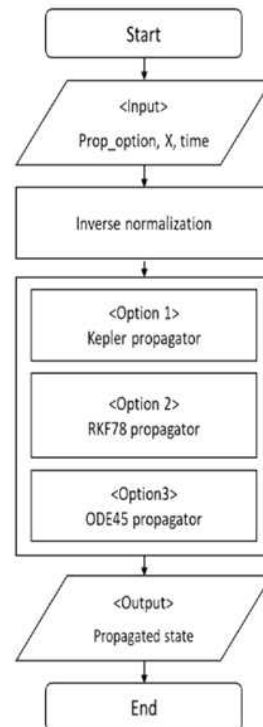
⑥ dyn_model

위성의 동역학 모델 (pkepler)를 이용하여 궤도 전파를 시키는 함수이다. pkepler 이외에도 runge-kutta 및 ode를 이용하여 다음과 같이 3가지의 방법을 적용할 수 있다. 이 때 출력된 값은 nominal state으로 궤도 전파하여 최종적으로 얻은 관측시간대별 위치, 속도이다.

```

39
40 function y_dyn=dyn_model(option,Xn,epoch_3D,utl_3D)
41 %#eml
42 opt_dyn=option;
43 epoch_t_sec=epoch_3D*24*3600;
44 time_sec=utl_3D*24*3600; sec 단위로 변환
45
46 Xn=Inverse_Normalize(Xn); % m, m/s 기존의 단위로 변환
47
48 if opt_dyn == 1 % option=1: use pkepler
49
50
51     ro=Xn(1:3)'; ro=ro/1000; % km
52     vo=Xn(4:6)'; vo=vo/1000; % km/s
53     dtsec=time_sec-epoch_t_sec;
54
55     [r,v]=pkepler(ro, vo, dtsec, 0, 0); % km, km/s
56
57     r=r'*1000; % m 1. pkepler를 사용하여 동역학 모델 궤도전파
58     v=v'*1000; % m/s (ri, rj, rk, vi, vj, vk, time)
59
60
61 elseif opt_dyn == 2 % option=2: use rkf78 fc
62
63     deq='f'; neq=length(Xn);
64     ti=epoch_t_sec; tf=time_sec;
65     h=(tf-ti)/12; tetol=1e-2;
66     Xn=Xn/1000; % km, km/s 2. runge-kutta 78을 사용하여 궤도전파
67
68     [xout]=rkf78(deq,neq,ti,tf,h,tetol,Xn);
69
70     r=xout(1:3,end)*1000; % m
71     v=xout(4:6,end)*1000; % m/s
72
73
74 elseif opt_dyn == 3
75
76     ti=epoch_t_sec; tf=time_sec;
77     options=odeset('reltol',1e-12);
78     Xn=Xn/1000; % km, km/s 3. ode45를 사용하여 궤도전파
79
80     [tX,xout]=ode45('ECIdyn32',[ti tf],Xn,options);
81
82     r=xout(end,1:3)*1000; % m
83     v=xout(end,4:6)*1000; % m/s
84
85 end
86
87 y_dyn=[r; v]; % m, m/s 관측 시간 별 [ri, rj, rk, vi, vj, vk] 출력
88
89
90 end

```



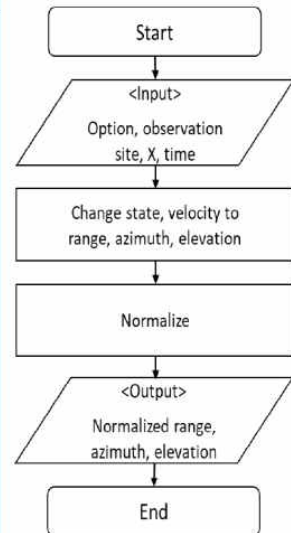
⑦ mea_model

mea_model.m 파일은 measurement모델 함수로 위치, 속도, 관측 site 정보를 이용하여 관측 값을 계산하는 함수이다. 이 때 필요한 자전축과 measurement모델 초기 값을 입력하였고 measurement모델 rv2razel을 통해 궤도 전파하여 생성된 시간대별 $X(t)$ 를 (ρ, β, el) 로 변환하여 표현할 수 있다.

```

편집기 - C:\Users\이진진\Desktop\analyze\어지\mea_model.m
mea_model.m
function yc=mea_model(option,obs_site,y_dyn,jdut1,ttt) 시간
% Observations from Kaena Point on January 29, 1995 for GEOS-III
% Rotation axis correction
% Constant for measurement model
% inertial input 입력
% rv->razel을 이용
% option=1: use rv2razel as measurement model
% 동역학 모델로 전파한 [ri, rj, rk, vi, vj, vk]를
% rd와 vdo에 관한 함수로 지정
% 위도, 경도, 고도를 각각 site(n)으로 지정
% rv->razel을 이용하여 ρ, β, el을 출력
% Normalize
% 시간대 별 ρ, β, el로 변환한 관측값을 yc로 출력
end

```

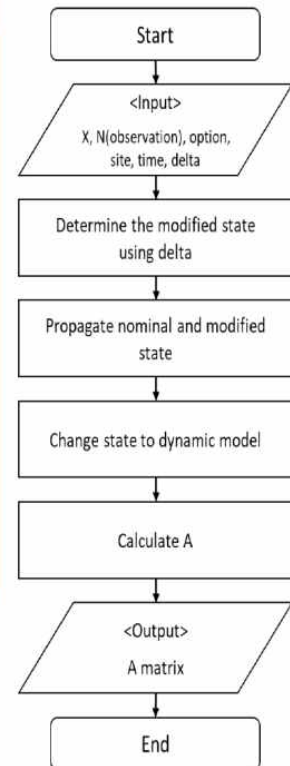


⑧ CalA

patrial derivative matrix [A]를 계산하는 함수이다. 계산하기 위해 필요한 element를 입력하고 $X \pm \delta$ 를 계산하여 numerical 방법인 central differencing method를 이용한다. 구한 값을 동역학 모델과 관측 모델에 적용하여 값을 산출한다. 반복적으로 계산하면 A matrix의 어림잡아 근사한 값을 계산할 수 있다.

```

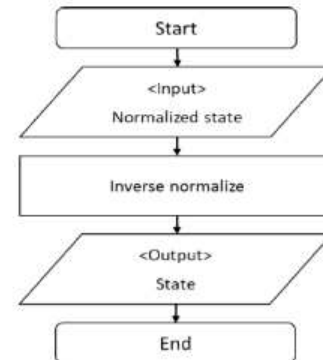
65 function [ A ] = CalA(Xn,nobp,opt_dyn,opt_mea,site,t,epoch_t,ttt...
66     ,eta,flag)    A matrix를 계산하기 위한 Input 입력
67 %=#eml
68 % ----- eta=0.01, and using central differencing by default ----- %
69 if nargin==8, flag=1; eta=0.01; end
70 if nargin==9, flag=1; end    nargin 함수의 입력 인수의 개수를 지정
71
72 A=zeros(nobp,length(Xn));
73
74 for j=1:length(Xn)
75
76     % ----- Determine the modified state ----- %
77     Xm1=Xn;
78     Xm2=Xn;
79     Xm1(j)=Xm1(j)+eta*Xn(j);    Delta를 이용하여 modified state를 결정
80     Xm2(j)=Xm2(j)-eta*Xn(j);
81
82     % ----- Get the computed observation ----- %
83     Xmp1=dyn_model(opt_dyn,Xm1,epoch_t,t);    전파한 nominal과 modified state를
84     Xmp2=dyn_model(opt_dyn,Xm2,epoch_t,t);    동역학모델에 적용하여 값을 계산
85     Xnp=dyn_model(opt_dyn,Xn,epoch_t,t);
86     obs_m1=mea_model(opt_mea,site,Xmp1,t,ttt);
87     obs_m2=mea_model(opt_mea,site,Xmp2,t,ttt);
88     obs_n=mea_model(opt_mea,site,Xnp,t,ttt);
89
90     % ----- Approximate the A matrix for single time ----- %
91     if flag==1;
92         A(1:nobp,j)=(obs_m1-obs_m2)./(2*eta*Xn(j));    각각에 대해 A matrix를 근사
93     elseif flag==2;
94         A(1:nobp,j)=(obs_m1-obs_n)./(eta*Xn(j));
95     end
96 end
  
```



⑨ Inverse_Normalize

' ⑤normalize ' 에서 계산한 값을 역으로 변환하여 실제 위치 속도 값으로 변환하는 함수이다. 과정은 normalize와 같으며 나누기 대신 곱하기를 사용하여 역변환하였다.

```
편집기 - C:\Users\W이진진\Downloads\궤도결정_속제2_codes\궤도결정
Inverse_Normalize.m
1 function y=Inverse_Normalize(x)
2
3 %% Re = 6378.137; % km
4 %% mu = 398600.4418; % km3/s2
5
6 Rem = 6378137; % m
7 mum = 3.986004418e14; % m3/s2
8
9 %% w=sqrt(mu/Re^3);
10 wm=sqrt(mum/Rem^3);
11
12 %% y=[x(1)*Re;
13 %%    x(2)*Re;
14 %%    x(3)*Re;
15 %%    x(4)*(Re*w);
16 %%    x(5)*(Re*w);
17 %%    x(6)*(Re*w)];
18
19 y=[x(1)*Rem;
20     x(2)*Rem;
21     x(3)*Rem;
22     x(4)*(Rem*wm);
23     x(5)*(Rem*wm);
24     x(6)*(Rem*wm)];
25
26 end
```



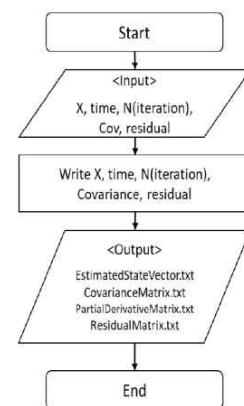
⑩ DifferentialCorrection_write

각 iteration에서 계산된 epoch time의 state와 covariance matrix, Partial Derivative matrix, 관측 값과 계산 값의 차이를 txt파일로 작성하여 저장한다. 최종적으로 EstimateState Vector.txt, CovarianceMatrix.txt, PartialDerivative.txt, ResidualMatrix.txt 파일이 출력된다.

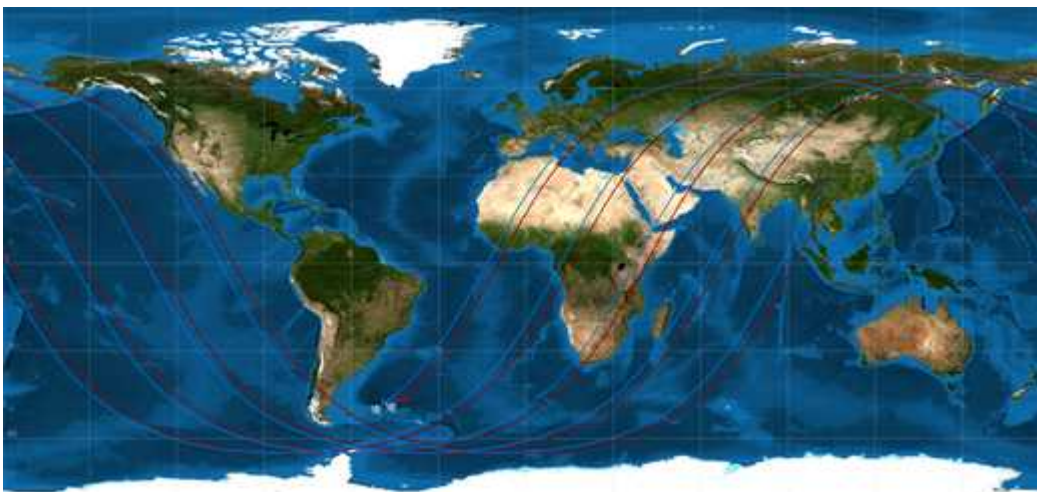
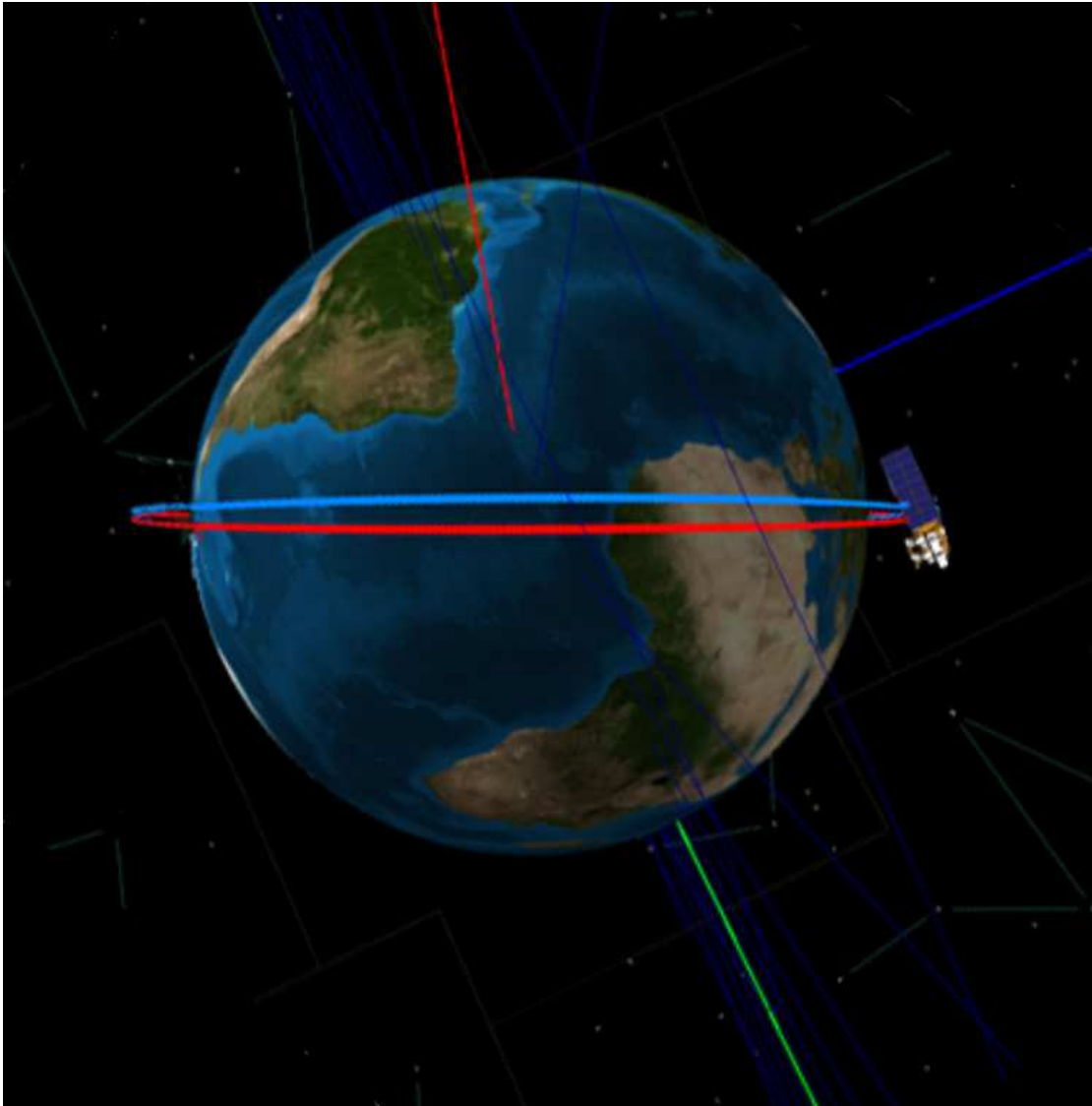
```

1  편집기 - C:\Users\W이진진\Desktop\analyzew\DifferentialCorrection_write.m
2  CalA.m DifferentialCorrection.m DifferentialCorrection_write.m +
3  function y=DifferentialCorrection_write(X,epoch_3D,iterN,rmsdel,P,a,b)
4      Differential Correction을 위해 필요한element를 입력
5      fid1=fopen('EstimatedStateVector.txt','w');
6      if fid1 == -1
7          fprintf('CANNOT MAKE THE RESULT FILE: EstimatedStateVector.txt')
8      end
9      fprintf(fid1,'The number of Iteration:\n'); 반복 횟수 (N)
10     fprintf(fid1,'%i\n\n',iterN);
11     fprintf(fid1,'The percentage for the RMS:\n'); △RMS
12     fprintf(fid1,'%3.3e\n\n',rmsdel);
13     fprintf(fid1,'Epoch time in Julian date:\n'); epoch time (Julian Date)
14     fprintf(fid1,'%f\n\n',epoch_3D);
15     fprintf(fid1,'Estimated state vector:\n');
16     fprintf(fid1,['r1; r2; r3; v1; v2; v3;'] { unit: normalized by Re, angular velocity of Earth }\n\n');
17     %% dlmwrite('EstimatedStateVector.txt',X,-append'); 위치와 속도 벡터 (Unit: Re, ω로 normalize된 값)
18     save('EstimatedStateVector.txt','X','-ascii','-append');
19     fclose(fid1); 해당어리즘의 파일로 저장
20
21     fid2=fopen('CovarianceMatrix.txt','w');
22     if fid2 == -1
23         fprintf('CANNOT MAKE THE RESULT FILE: CovarianceMatrix.txt')
24     end
25     fprintf(fid2,'Covariance matrix:\n\n');
26     %% dlmwrite('CovarianceMatrix.txt',P,'delimiter','\t','precision',5,-append');
27     save('CovarianceMatrix.txt','P','-ascii','-append');
28     fclose(fid2);
29     Covariance matrix를 생성하여 P 값을 기록
30
31     fid3=fopen('PartialDerivativeMatrix.txt','w');
32     if fid3 == -1
33         fprintf('CANNOT MAKE THE RESULT FILE: PartialDerivativeMatrix.txt')
34     end
35     fprintf(fid3,'Partial derivative matrix:\n\n');
36     %% dlmwrite('PartialDerivativeMatrix.txt',a,'delimiter','\t','precision',5,-append');
37     save('PartialDerivativeMatrix.txt','a','-ascii','-append');
38     fclose(fid3);
39     Partial Derivative Matrix [A]를 생성하여 a 값을 기록
40
41     fid4=fopen('ResidualMatrix.txt','w');
42     if fid4 == -1
43         fprintf('CANNOT MAKE THE RESULT FILE: ResidualMatrix.txt')
44     end
45     fprintf(fid4,'Residual matrix:\n\n');
46     %% dlmwrite('ResidualMatrix.txt',b,'delimiter','\t','precision',5,-append');
47     save('ResidualMatrix.txt','b','-ascii','-append');
48     fclose(fid4);
49     Residual Matrix(r)를 생성하여 b 값을 기록
50
51     %% xlswrite('EstimatedStateVector',Xn);
52     %% xlswrite('CovarianceMatrix',P);
53     %% xlswrite('PartialDerivativeMatrix',a);
54     %% xlswrite('ResidualMatrix',b);
55
56     end

```



9) Plot the initial orbit and the final orbit by using the STK or GMAT
(IOD : RED / POD : BLUE)



10) Explain the POD results.

Iteration	r_1	r_2	r_3	v_1	v_2	v_3
IOD	6094.69Km	2811.93Km	2631.66Km	3.62Km/s	-2.04Km/s	-6.19Km/s
1	5751.62Km	2676.01Km	3462.67Km	4.45Km/s	-1.89Km/s	-5.94Km/s
2	5748.50Km	2679.52Km	3443.02Km	4.33Km/s	-1.92Km/s	-5.73Km/s
3	5748.62Km	2679.65Km	3442.76Km	4.33Km/s	-1.92Km/s	-5.72Km/s
POD	5748.62Km	2679.65Km	3442.76Km	4.33Km/s	-1.92Km/s	-5.72Km/s
POD- IOD	346.07Km	132.28Km	-811.1Km	-0.71Km/s	-0.12Km/s	-0.47Km/s

총 4회의 Iteration에 대해 수행하여 POD를 계산하였고 위와 같은 데이터를 얻을 수 있었다. Weighted Matrix에서 가중치의 변화는 없는 것으로 가정하였으며 반복을 진행할수록 매우 미세하게 변화하다가 한 점에 수렴하는 값을 유지하는 것을 볼 수 있었다. Iteration을 통해 POD를 계산해본 결과, 생각보다 Initial 값과 큰 차이를 보였으며 위성의 정확한 궤도를 계산하기 위해서는 위와 같은 과정을 반드시 거쳐야 하는 것을 알 수 있었다.